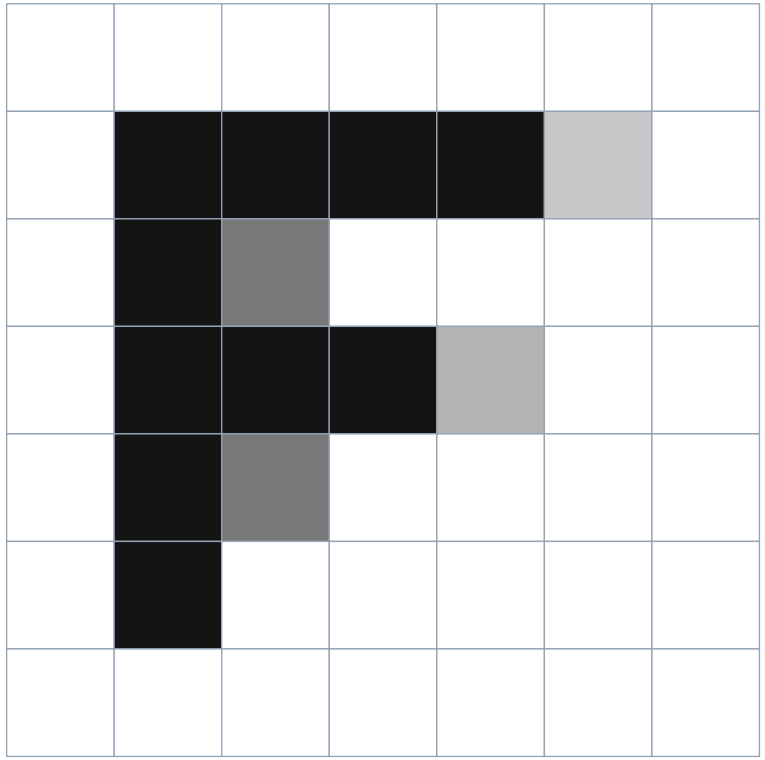
Estrategias de diseño de algoritmos

Clase 19 · Jueves 12 de noviembre de 2026 · Aula

Fuerza bruta	prueba todo
Divide y vencerás	parte y combina
Recursividad	se llama a sí misma
Programación dinámica	recuerda lo calculado
Voraz	lo mejor ahora
Backtracking	prueba y regresa

Para la computadora, una imagen son números

Antes de la noticia: cada píxel es una casilla de una matriz. En escala de grises, 0 = negro, 255 = blanco y lo de en medio, grises.



Lo que ves

255	255	255	255	255	255	255
255	20	20	20	20	200	255
255	20	120	255	255	255	255
255	20	20	20	180	255	255
255	20	120	255	255	255	255
255	20	255	255	255	255	255
255	255	255	255	255	255	255

Lo que guarda la computadora: lista_imagen

Una foto de celular

12 megapíxeles = **12 millones** de casillas. A color son 3 números por píxel (rojo, verde, azul): **36 millones** de números.

En Python

Una lista de listas: `lista_imagen[1][5]` es el píxel de la fila 1, columna 5 → `200`, un gris claro.

Editar una foto = **hacer cuentas con esos números**. Oscurecer: multiplicar **todos** por 0.5.

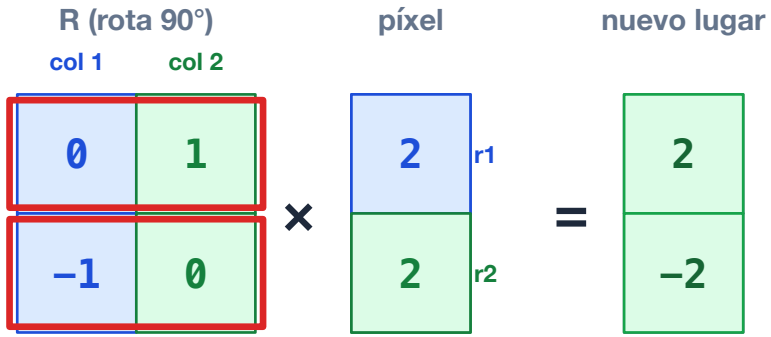
Rotar la imagen 90° = multiplicar por una matriz

Cada píxel tiene coordenadas (x, y) desde el centro. Multiplicar (x, y) por la matriz R lo lleva a su lugar en la imagen rotada.

y ↑ \ x →	−3	−2	−1	0	1	2	3
3	255	255	255	255	255	255	255
2	255	20	20	20	20	200	255
1	255	20	120	255	255	255	255
0	255	20	20	20	180	255	255
−1	255	20	120	255	255	255	255
−2	255	20	255	255	255	255	255
−3	255	255	255	255	255	255	255

Original

El píxel **200** está en **x = 2, y = 2**.



Mismo color = se multiplican: la **columna 1** de R con el **renglón 1** del píxel (x) y la **columna 2** con el **renglón 2** (y).

renglón 1: 0 × 2 + 1 × 2 = 2

renglón 2: −1 × 2 + 0 × 2 = −2

El 200 pasa de **(2, 2)** a **(2, −2)**: la esquina de arriba ahora está abajo.

y ↑ \ x →	−3	−2	−1	0	1	2	3
3	255	255	255	255	255	255	255
2	255	20	20	20	20	20	255
1	255	255	120	20	120	20	255
0	255	255	255	20	255	20	255
−1	255	255	255	180	255	20	255
−2	255	255	255	255	255	200	255
−3	255	255	255	255	255	255	255

Rotada 90° a la derecha

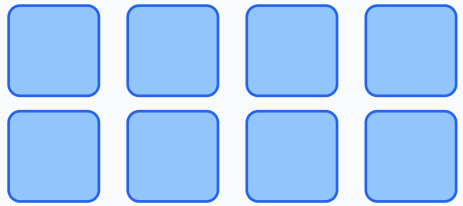
La misma multiplicación para los 49 píxeles, y ninguno espera a otro. Una pantalla de 1920 × 1080 tiene **2 millones** de píxeles, 60 veces por segundo; un juego 3D además gira cada punto de cada objeto con matrices así. Hacerlo **todo a la vez** es el trabajo de una GPU.

CPU vs. GPU: por qué NVIDIA nació en los videojuegos

La CPU hace cualquier cosa, una tras otra. La GPU hace la misma cuenta sencilla sobre millones de datos a la vez.

CPU · Central Processing Unit

Unidad central de procesamiento: el procesador de siempre.

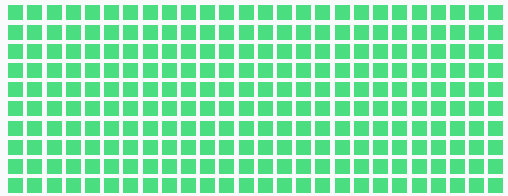


8 a 24 núcleos, muy rápidos y listos para cualquier tarea, pero hacen las cosas casi **una tras otra**.

Como **8 chefs expertos**: cocinan cualquier platillo.

GPU · Graphics Processing Unit

Unidad de procesamiento gráfico: la tarjeta de video.



Miles de núcleos sencillos (más de 16 mil en una tarjeta actual) haciendo **la misma cuenta a la vez**.

Como **10,000 ayudantes que sólo saben picar**: para picar 10,000 cebollas, ganan ellos.

1993
Nace NVIDIA
para gráficos de videojuegos

→

1999
GeForce 256
la primera tarjeta llamada «GPU»

→

2006
CUDA
Compute Unified Device Architecture: programar la GPU para cualquier cálculo

→

2012
AlexNet
red neuronal entrenada en 2 GPUs de videojuegos

→

2022 →
ChatGPT
centros de datos con decenas de miles de GPUs

Los videojuegos pagaron el desarrollo de un chip para **multiplicar matrices en paralelo**. Años después, eso resultó ser justo lo que necesita la IA.

Las palabras también son números: embeddings

Un modelo de lenguaje convierte cada palabra en un vector. Ejemplo de juguete con 4 columnas que elegimos nosotros.

palabra	realeza	juventud	masculino	femenino
hombre	0	0	**1**	0
mujer	0	0	0	**1**
niño	0	**1**	**1**	0
niña	0	**1**	0	**1**
rey	**1**	0	**1**	0
reina	**1**	0	0	**1**
príncipe	**1**	**1**	**1**	0
princesa	**1**	**1**	0	**1**
realeza	**1**	0	0	0

realeza no es una persona: es el rasgo que queda al hacer rey – hombre.

```
   rey        [1, 0, 1, 0]
−  hombre     [0, 0, 1, 0]
=  realeza    [1, 0, 0, 0]
```

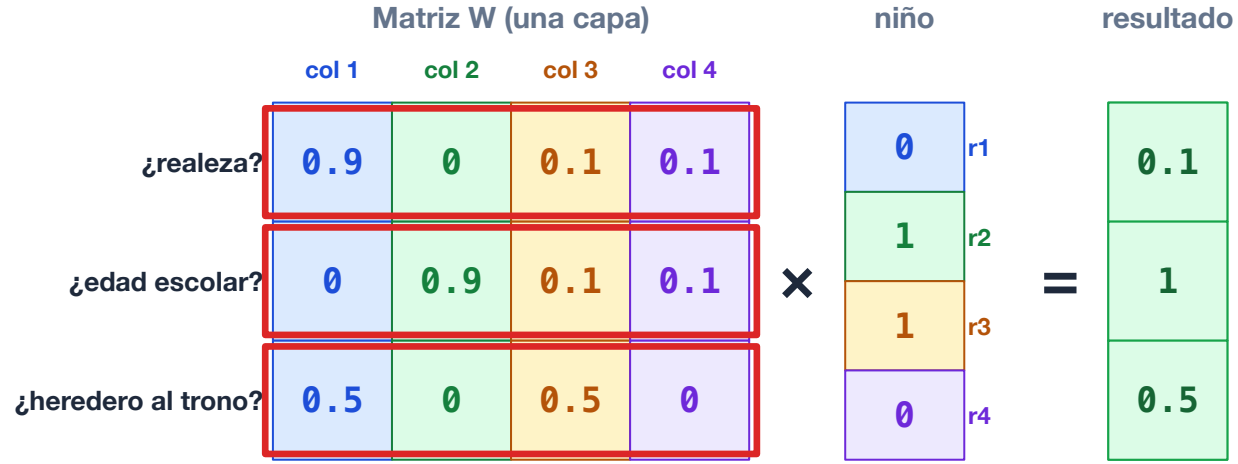
```
   niño       [0, 1, 1, 0]
+  realeza    [1, 0, 0, 0]
=  príncipe   [1, 1, 1, 0]
```

```
   rey        [1, 0, 1, 0]
−  hombre     [0, 0, 1, 0]
+  mujer      [0, 0, 0, 1]
=  reina      [1, 0, 0, 1]
```

En un modelo real: cada palabra tiene **miles** de columnas con decimales (p. ej. `[0.12, −0.83, 0.41, …]`) y nadie las elige: **el modelo las aprende**.

Lo que hace un modelo de lenguaje: multiplicar matrices

Lo mismo que con la rotación: cada renglón de W hace una «pregunta» sobre la palabra «niño» y la responde con un número.



Mismo color = se multiplican: columna k del renglón × renglón k del vector.

renglón 1: `0.9×0` + `0×1` + `0.1×1` + `0.1×0` = `0.1`

renglón 2: `0×0` + `0.9×1` + `0.1×1` + `0.1×0` = `1`

renglón 3: `0.5×0` + `0×1` + `0.5×1` + `0×0` = `0.5`

Python

```python
def multiplicar_matriz_vector(lista_matriz, lista_vector):
    lista_resultado = []
    for lista_fila in lista_matriz:
        total = 0
        for i in range(len(lista_fila)):
            total = total + lista_fila[i] * lista_vector[i]
        lista_resultado.append(total)
    return lista_resultado
```

Es un **acumulador**: `lista_fila[i] * lista_vector[i]` es «columna i × renglón i».

¿Cómo se lee el resultado? Cada renglón de W hace una «pregunta» sobre la palabra. Para niño: realeza **0.1**, edad escolar **1**, heredero al trono **0.5** (es hombre, pero no de la realeza). Príncipe daría **[1, 1, 1]**.

Los renglones no dependen unos de otros: la GPU los calcula todos a la vez, igual que los píxeles de la imagen. Un modelo con 70 mil millones de números en sus matrices hace ≈ **140 mil millones** de multiplicaciones y sumas **por cada palabra** que escribe. Sin GPUs, una respuesta de ChatGPT tardaría muchísimo más.

¿Y qué son los «pesos» de un modelo?

Son los números dentro de esas matrices. Nadie los escribe a mano: el entrenamiento los encuentra.

Rotar una imagen

Los números de la matriz los **eligimos nosotros**: `[[0, 1], [−1, 0]]`.

Modelo de IA

Los números **empiezan al azar** y el entrenamiento los ajusta poco a poco, comparando lo que predice con millones de textos.

Entrenar

Encontrar los pesos: **meses, miles de GPUs** y cientos de millones de dólares.

Usar el modelo

Multiplicar por esos pesos cada vez que escribe una palabra. **También con GPUs.**

Al final, los pesos son un archivo

70 mil millones de números × 2 bytes ≈ **140 GB**. Con ese archivo y suficientes GPUs, cualquiera corre el modelo. **Publicar o no ese archivo** es la diferencia entre **pesos abiertos y cerrados**, y **tener o no las GPUs** decide quién puede entrenar y usar los modelos. Con esto en mente, la noticia de hoy.

Noticia tech

Jensen Huang deja de ser CEO de NVIDIA, la empresa que fundó en 1993 y que hace las GPUs con las que se entrena la IA.

1993 – 2026

Fundador y único CEO

Convirtió a NVIDIA de una empresa de tarjetas para videojuegos en la proveedora de casi todas las GPUs con las que se entrena la IA.

9 – 11 NOV

El escándalo

Se reportan relaciones personales; una, con Alice Huang, ejecutiva a la que el New York Times vincula con el Ministerio de Seguridad del Estado de China. Investigan el FBI y el Congreso.

12 NOV

La renuncia

Con efecto inmediato. Colette Kress (directora de finanzas) queda como CEO interina. Huang no seguirá en NVIDIA en ningún puesto.

17 NOV

Lo que sigue

NVIDIA presenta sus resultados trimestrales. Huang dijo estar dispuesto a declarar ante el Congreso.

Cómo reaccionó la bolsa ese día

NVIDIA

hasta –16 %

y luego ≈ –5 %

AMD

≈ +3 a +7 %

rival directo

Broadcom

≈ +5 %

chips a la medida

TSMC · Intel

≈ –1 % · –4 %

arrastrados por NVIDIA

Índices

≈ sin cambio

Dow, S&P 500, Nasdaq

El producto no cambió de un día para otro; lo que cambió es la incertidumbre: quién decide, qué pasará con las investigaciones y con las ventas a China. Por eso suben los rivales: AMD y Broadcom son el «plan B» de quien compra chips.

Cifras intradía del 12 nov 2026 (pueden cambiar al cierre). Las acusaciones de espionaje no están probadas; China las niega. Fuentes: 8-K de NVIDIA, CNBC, Al Jazeera, NBC, Axios, Yahoo Finance.

Noticia tech · ¿Por qué le interesaría NVIDIA al gobierno chino?

Para entrenar IA de frontera se necesitan miles de GPUs, y las mejores las vende una sola empresa.

Entrenar un modelo de IA de frontera → **Decenas de miles de GPUs** → **Casi todas son de NVIDIA (+ CUDA)** → **EE.UU. limita venderlas a China**

1 · Los chips son el cuello de botella

Sin GPUs no hay modelos grandes. Un país puede tener talento y datos, pero si no consigue chips, se queda atrás.

2 · CUDA: el «idioma» de las GPUs

Casi todo el software de IA está escrito para CUDA, de NVIDIA. Cambiar de chip implica reescribir y volver a optimizar.

3 · Un cliente enorme

China llegó a representar al menos una quinta parte de las ventas de NVIDIA para centros de datos.

4 · Lo que se reporta

El objetivo habría sido conocer a Huang e influir en su postura sobre vender chips a China. Se vincula a Alice Huang con el paso de ≈ 240 mil chips restringidos (≈ 7 mil millones de dólares).

Una hipótesis: influir en quien decide qué chips se venden vale más que robar un diseño. Un diseño, sin la fábrica (TSMC, en Taiwán) y sin CUDA, sirve de poco.

Fuentes: New York Times (vía NBC y Axios), The Information (vía Business Standard), CNBC. El consejo de NVIDIA dice no tener razones para creer que se filtraron secretos comerciales.

Noticia tech · Pesos abiertos vs. pesos cerrados

Recuerda: los pesos son los números de las matrices del modelo. Quien tiene el archivo (y GPUs), puede correr el modelo.

China: pesos abiertos

DeepSeek · Qwen (Alibaba) · Kimi (Moonshot) · GLM (Zhipu) · MiniMax

- **Con menos chips, compiten por adopción:** si todo el mundo usa tu modelo, tú pones el estándar.
- **El negocio está en la nube** y en los servicios (p. ej. Alibaba Cloud), no en vender el modelo.
- **La comunidad mundial** lo mejora y lo adapta gratis.
- **Precio:** modelos cercanos a la frontera por una fracción del costo.

EE.UU.: pesos cerrados

OpenAI (GPT) · Anthropic (Claude) · Google (Gemini)

- **Recuperar la inversión** enorme en cómputo cobrando por usar el modelo (API).
- **Controlar el uso:** seguridad y abuso; no se puede quitar un filtro a un modelo que no tienes.
- **Conservar la ventaja:** lo que no se publica no se copia.
- Hay excepciones: Meta (Llama), OpenAI (gpt-oss) y NVIDIA (Nemotron) publican modelos abiertos.

¿Y qué tiene que ver con la noticia?

Los pesos se pueden descargar; los chips no. Un modelo abierto no sirve sin GPUs para entrenarlo y correrlo. Si EE.UU. controla los chips, la estrategia china depende de conseguir hardware (comprarlo o fabricarlo, como Huawei) y de **diseñar mejores algoritmos**: DeepSeek se hizo famoso por entrenar con muchos menos chips. Con poco hardware, el algoritmo es la ventaja.

Objetivo de la sesión

Reconocer seis formas de atacar un problema y saber cuándo conviene cada una.

1 · Fuerza bruta

Prueba todas las posibilidades y revisa cada una.

Piensa en: Probar todas las combinaciones de un candado.

2 · Divide y vencerás

Parte el problema en pedazos más chicos, resuelve cada uno y combina.

Piensa en: Buscar una palabra en el diccionario abriéndolo a la mitad.

3 · Recursividad

Una función que se llama a sí misma con un problema más chico.

Piensa en: Muñecas rusas: dentro de cada una hay otra más pequeña.

4 · Programación dinámica

Guarda lo que ya calculaste para no volver a calcularlo.

Piensa en: Anotar los resultados en una tabla en lugar de repetir cuentas.

5 · Voraz

En cada paso elige lo que se ve mejor ahora, sin regresar.

Piensa en: Dar cambio empezando por la moneda más grande.

6 · Backtracking

Avanza; si un camino no sirve, regresa y prueba otro.

Piensa en: Salir de un laberinto.

1 de 6

Fuerza bruta

Genera todos los candidatos y revisa uno por uno.

Fuerza bruta: probar todos los pares

Regalo de intercambio: ¿qué dos productos suman exactamente el presupuesto?

Python

```python
def encontrar_par_presupuesto(lista_precios, presupuesto):
    for i in range(len(lista_precios)):
        for j in range(i + 1, len(lista_precios)):
            if lista_precios[i] + lista_precios[j] == presupuesto:
                return [lista_precios[i], lista_precios[j]]
    return []
```

Llamada

```
encontrar_par_presupuesto(
    [300, 200, 550, 450], 750)
```

Pares que revisa

`300 + 200`	= 500 ✗
`300 + 550`	= 850 ✗
`300 + 450`	**= 750 ✓ regresa [300, 450]**

¿Cuántos pares hay que revisar en el peor caso?

10 productos	1,000 productos	1,000,000 productos
45 pares	**≈ 500 mil pares**	**≈ 500 mil millones de pares**

Con n productos son n·(n−1)/2 pares: **O(n²)**. Dos ciclos anidados que recorren la misma lista suelen ser la huella de la fuerza bruta.

¿Cuándo sí sirve la fuerza bruta?

Depende de cuántos candidatos haya. Cuando crecen exponencialmente, deja de ser opción.

Candado de 3 dígitos

1,000

combinaciones

Se prueban en minutos a mano.

Contraseña de 8 caracteres (letras y números)

$62^8 \approx 218$ billones

combinaciones

≈ 2.5 días a mil millones de intentos por segundo.

Contraseña de 12 caracteres

$62^{12} \approx 3.2 \times 10^{21}$

combinaciones

≈ 100 mil años a la misma velocidad.

Úsala cuando…

- hay pocos candidatos;
- necesitas una primera versión correcta para comparar contra la versión rápida;
- la exactitud importa más que el tiempo.

Evítala cuando…

- los candidatos se multiplican con cada dato nuevo (combinaciones, ordenamientos, subconjuntos);
- hay una forma de descartar candidatos sin revisarlos (eso es lo que hacen las otras cinco estrategias).

Fuerza bruta + hardware: por eso las GPUs también sirven para romper contraseñas y por eso te piden contraseñas largas.

2 de 6

Divide y vencerás

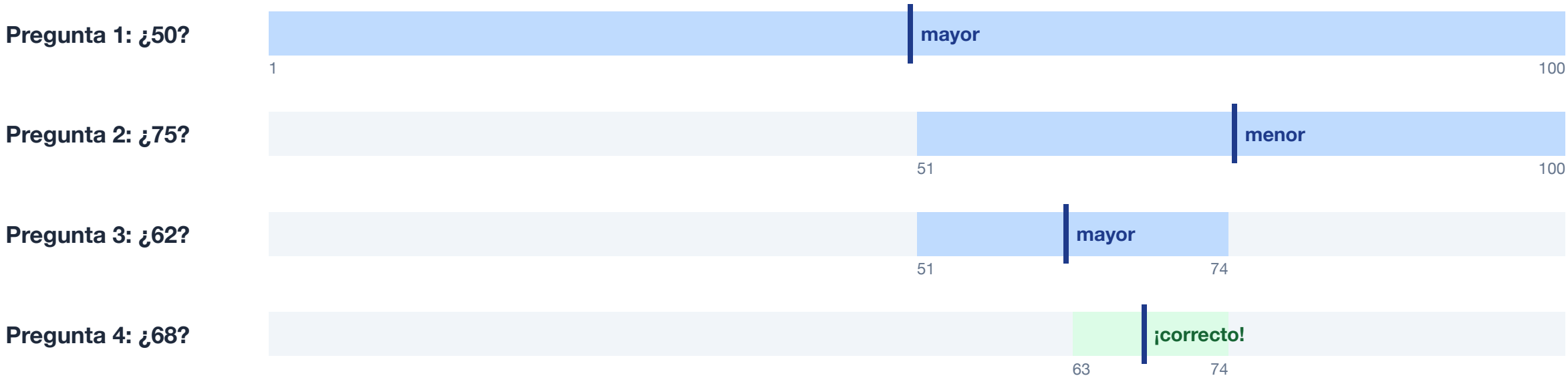
Parte el problema en pedazos más chicos, resuelve cada uno y combina.

Divide y vencerás: adivina el número

Pienso un número del 1 al 100. Tú preguntas; yo sólo contesto «mayor», «menor» o «¡correcto!».

Dividir → **Vencer** → **Combinar**

Aquí: pregunta por la mitad → quédate con la mitad donde está el número → repite.



Fuerza bruta: hasta 100 preguntas. **Divide y vencerás:** máximo 7, porque $2^7 = 128$. **Con un millón de números:** máximo 20. Cada pregunta descarta la mitad: **O(log n)**.

Búsqueda binaria: el juego en Python

Regresa la posición del objetivo en una lista ordenada, o –1 si no está.

Python

```python
def buscar_posicion_binaria(lista_ordenada, objetivo):
    izquierda = 0
    derecha = len(lista_ordenada) - 1
    while izquierda <= derecha:
        mitad = (izquierda + derecha) // 2
        if lista_ordenada[mitad] == objetivo:
            return mitad
        elif lista_ordenada[mitad] < objetivo:
            izquierda = mitad + 1
        else:
            derecha = mitad - 1
    return -1
```

Requisito

La lista debe estar **ordenada**. Si no, al descartar una mitad podrías tirar justo lo que buscas.

Por qué es rápida

Cada vuelta del while descarta la mitad de lo que queda: **O(log n)** contra **O(n)** de revisar uno por uno.

Otro ejemplo: merge sort

Parte la lista en mitades, ordena cada mitad y **combina** las dos mitades ordenadas. Lo vimos en octubre.

Patrón: el problema se parte en pedazos del mismo tipo, pero más chicos. Por eso divide y vencerás casi siempre se escribe con recursividad.

3 de 6

Recursividad

Una función que se llama a sí misma con un problema más chico.

Recursividad: una función que se llama a sí misma

Factorial: 4! = 4 × 3 × 2 × 1 = 24. Pero también 4! = 4 × 3!, y 3! = 3 × 2!…

Python

```python
1  def calcular_factorial(n):
2      if n == 0:
3          return 1
4      return n * calcular_factorial(n - 1)
```

Caso base · líneas 2 y 3

Cuándo **parar**: un problema tan chico que la respuesta se sabe sin volver a llamar. 0! = 1.

Caso recursivo · línea 4

Llamarse con un problema **más chico** (n – 1) que se acerca al caso base.

Se lee igual que la definición

```
4!    = 4 × 3!
   3!    = 3 × 2!
      2!    = 2 × 1!
         1!    = 1 × 0!
            0!    = 1   ← caso base
```

Truco para escribirla: supón que la función ya funciona para un problema más chico. ¿Cómo la usas para resolver el tuyo? Eso es el caso recursivo.

La pila de llamadas: calcular_factorial(4)

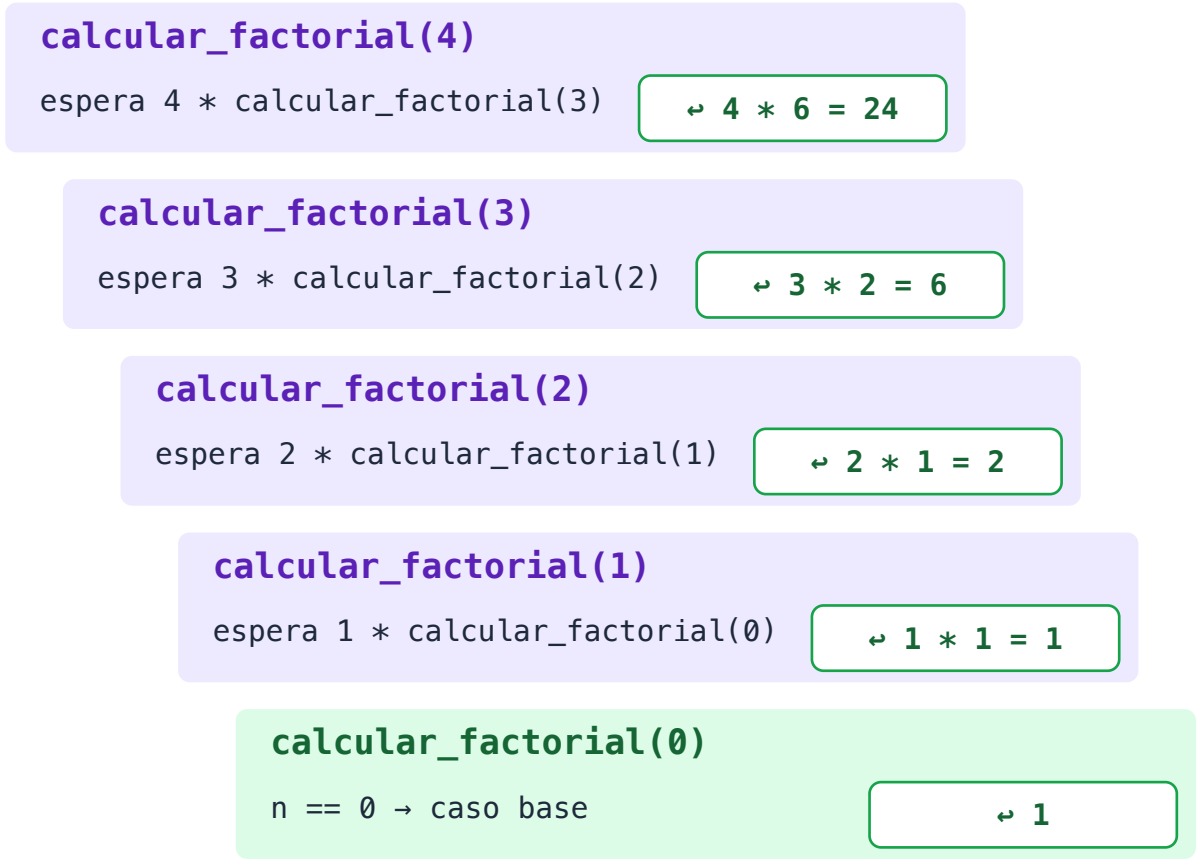
Cada llamada espera la respuesta de la siguiente. Cuando llega al caso base, las respuestas regresan.

Python

```python
def calcular_factorial(n):
    if n == 0:
        return 1
    return n * calcular_factorial(n - 1)
```

Bajando: cada llamada se queda «en pausa» esperando a la siguiente.

Subiendo: el caso base regresa 1 y cada llamada termina su multiplicación.



Los errores más comunes con recursividad

Corren sin error de sintaxis, pero nunca terminan (o terminan con RecursionError).

1 · Sin caso base

✗ Python

```python
def calcular_factorial(n):
    return n * calcular_factorial(n - 1)
```

Nunca para: Python se detiene con RecursionError después de unas mil llamadas pendientes.

2 · El problema no se hace más chico

✗ Python

```python
def calcular_factorial(n):
    if n == 0:
        return 1
    return n * calcular_factorial(n)
```

Se llama con el mismo n: nunca se acerca al caso base.

3 · Datos que nunca llegan al caso base

✗ Python

```python
calcular_factorial(-1)
```

Baja a –2, –3, –4… y nunca llega a 0. Solución: un caso base que también cubra estos datos (n <= 0) o validar la entrada.

Toda recursión se puede escribir con un ciclo. La recursión conviene cuando el problema se parte en pedazos del mismo tipo: divide y vencerás, carpetas dentro de carpetas, backtracking.

4 de 6

Programación dinámica

Guarda lo que ya calculaste para no volver a calcularlo.

El problema de repetir trabajo: Fibonacci

0, 1, 1, 2, 3, 5, 8, 13… Cada número es la suma de los dos anteriores.

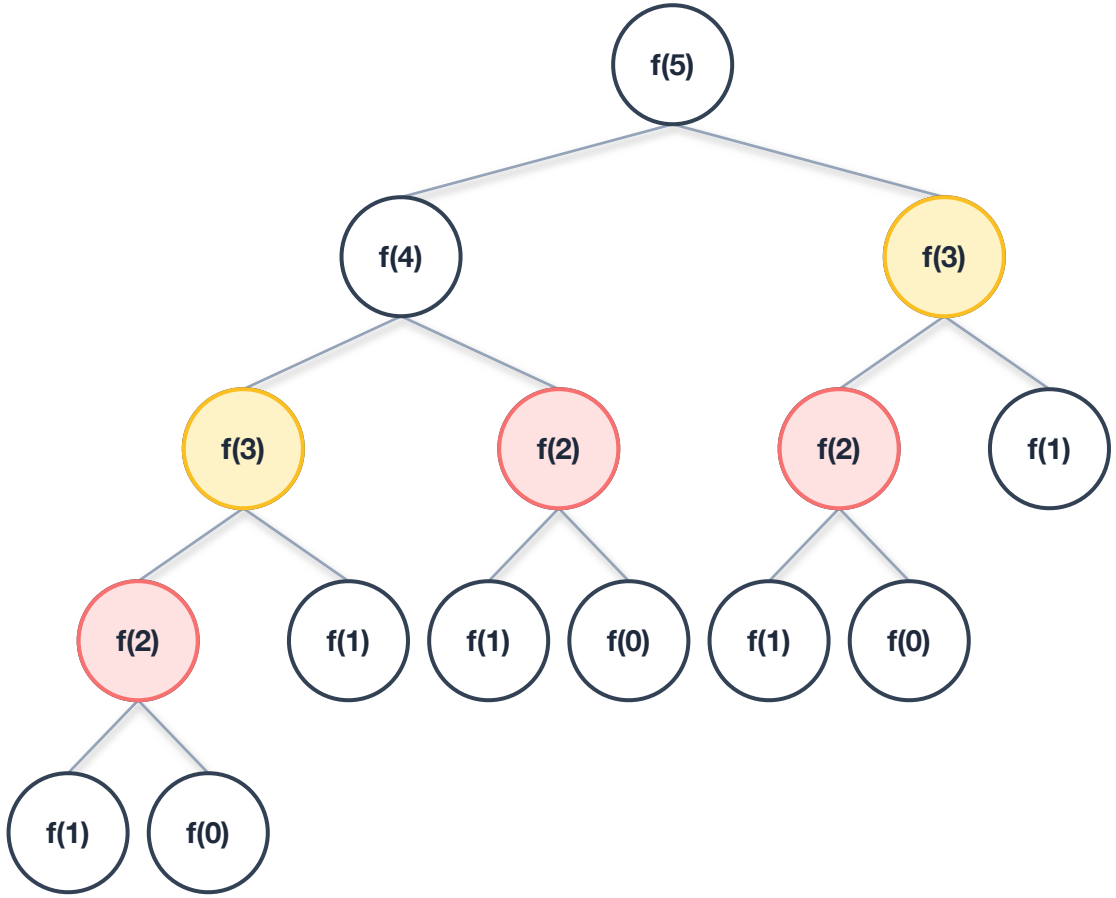
Python

```python
def calcular_fibonacci(n):
    if n <= 1:
        return n
    fib_anterior = calcular_fibonacci(n - 1)
    fib_antes_anterior = calcular_fibonacci(n - 2)
    return fib_anterior + fib_antes_anterior
```

¿Cuántas llamadas hace?

`f(5)`	15 llamadas
`f(10)`	177 llamadas
`f(30)`	**≈ 2.7 millones llamadas**
`f(40)`	**≈ 331 millones llamadas**

Crece como $\mathbf{2^n}$: cada llamada abre otras dos.



f(3) se calcula 2 veces · f(2) se calcula 3 veces

Programación dinámica: anota y reutiliza

Llena una tabla de los casos chicos a los grandes; cada casilla usa las que ya están llenas.

Python

```python
def calcular_fibonacci_tabla(n):
    lista_fib = [0, 1]
    for i in range(2, n + 1):
        lista_fib.append(lista_fib[i - 1] + lista_fib[i - 2])
    return lista_fib[n]
```

La regla de la tabla

Cada casilla = la de la izquierda + la de antes.

Las dos primeras (0 y 1) son los **casos base**: se escriben a mano.

`calcular_fibonacci_tabla(10)`

i = 0	i = 1	i = 2	i = 3	i = 4	i = 5	i = 6	i = 7	i = 8	i = 9	i = 10
0	1	1	2	3	5	8	13	21	34	55
		1+0	1+1	2+1	3+2	5+3	8+5	13+8	21+13	34+21

Recursiva — f(10) → **177** llamadas; f(40) → ≈ **331 millones** de llamadas — $O(2^n)$

Con tabla — f(10) → **9** sumas; f(40) → **39** sumas — $O(n)$

Cuándo usar programación dinámica

Dos condiciones, dos formas de escribirla y dónde la usas sin saberlo.

Condición 1 · Subproblemas que se repiten

Como f(3) y f(2) en el árbol. Si cada subproblema aparece una sola vez, anotarlo no ahorra nada.

Condición 2 · Lo mejor se arma con lo mejor de los pedazos

El camino más corto a tu casa pasa por el camino más corto a cada esquina intermedia.

De arriba hacia abajo (memoización)

La recursión de siempre, pero antes de calcular revisa si ya lo anotó y, al terminar, lo anota.

De abajo hacia arriba (tabla)

Llena la tabla desde los casos chicos hasta el que te piden, con un ciclo. Es lo que hicimos con Fibonacci.

La usas sin saberlo

Corrector ortográfico
¿Cuántas letras hay que cambiar para pasar de «algoritmo» a «algorimto»?

git diff
¿Qué líneas cambiaron entre dos versiones de tu código?

Biología
Comparar secuencias de ADN para encontrar parecidos.

5 de 6

Algoritmos voraces (greedy)

En cada paso elige lo que se ve mejor ahora y nunca regresa.

Voraz: dar cambio con la moneda más grande

¿Cómo das 37 pesos con el menor número de monedas de $10, $5, $2 y $1?

Python

```python
def calcular_monedas_cambio(cantidad, lista_monedas):
    lista_cambio = []
    for moneda in lista_monedas:
        while cantidad >= moneda:
            lista_cambio.append(moneda)
            cantidad = cantidad - moneda
    return lista_cambio
```

Llamada

```
calcular_monedas_cambio(37, [10, 5, 2, 1])
```

Falta por dar



Regresa **[10, 10, 10, 5, 2]**: 5 monedas.

Por qué es atractivo

Es rápido y simple: decide en cada paso sin probar combinaciones ni regresar.

Lo usas sin saberlo

Rutas en mapas (algoritmo de Dijkstra) y compresión de archivos (código de Huffman, en .zip) son voraces.

¿Lo voraz siempre funciona? No.

En un país con monedas de $4, $3 y $1, ¿cómo das $6 de cambio?

Llamada

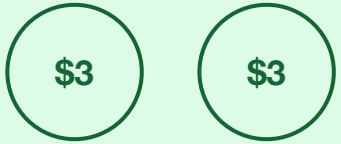
```
calcular_monedas_cambio(6, [4, 3, 1])
```

Voraz: [4, 1, 1] → 3 monedas



Toma 4 porque es la más grande… y ya no cabe ningún 3.

Lo mejor: [3, 3] → 2 monedas



Empezar con la moneda más grande no era lo mejor.

Lo voraz funciona sólo si elegir lo mejor de ahora nunca te cierra la mejor solución. Con las monedas de México sí se cumple; con $4, $3 y $1, no. Cuando no hay esa garantía, la **programación dinámica** revisa todas las opciones de cada subproblema.

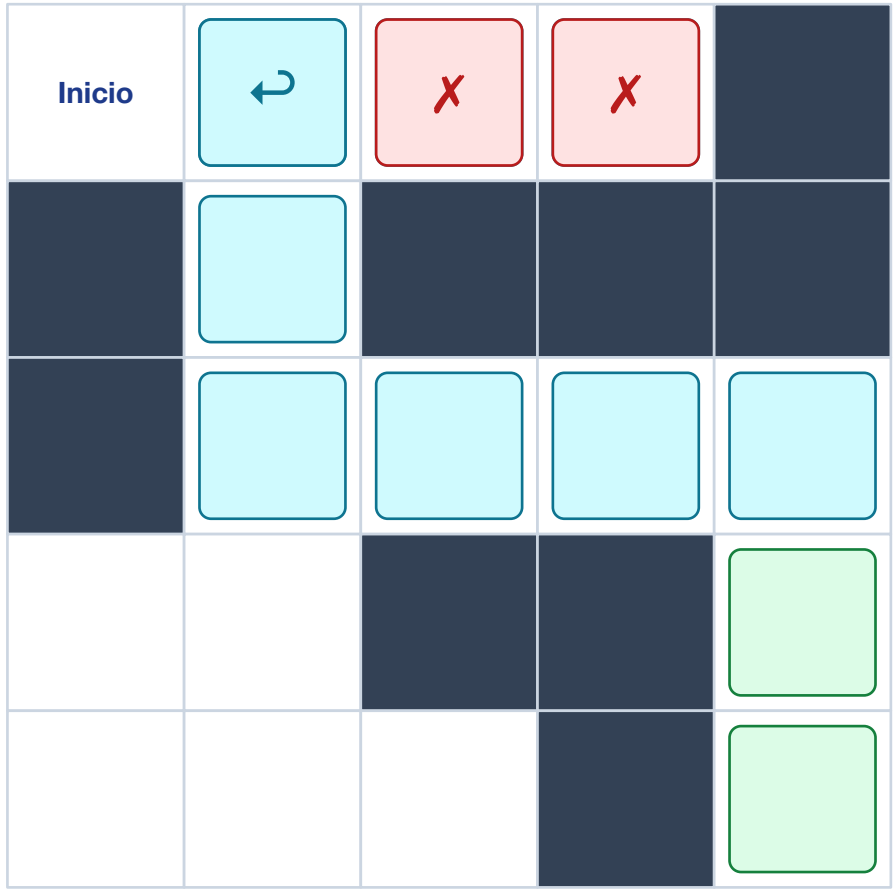
6 de 6

Backtracking

Avanza paso a paso; si un camino no sirve, regresa y prueba otro.

Backtracking: salir del laberinto

Regla: en cada casilla intenta derecha, abajo, izquierda y arriba, en ese orden. Si no puede avanzar, regresa.



1 · Avanza a la derecha hasta que hay pared.

2 · Callejón: derecha y abajo son pared; a la izquierda ya pasó.

3 · **Regresa** una casilla: tampoco hay otra opción. Regresa otra más…

4 · …en la segunda casilla sí puede bajar: baja dos.

5 · Ahora sí hay camino a la derecha, hasta el borde.

6 · Baja hasta la **salida**. ✓

La diferencia con fuerza bruta: no genera todos los caminos posibles; en cuanto uno choca, lo abandona y prueba otro.

La receta del backtracking

Elegir, explorar y deshacer. Casi siempre se escribe con recursividad.

Pseudocódigo

```
resolver(paso):
    si ya terminé: guardar la solución
    para cada opción posible en este paso:
        si la opción respeta las reglas:
            elegirla
            resolver(paso + 1)
            deshacerla        # ← backtrack
```

Poda

En cuanto una opción rompe una regla, **no explora nada de lo que seguiría**. Eso la hace mucho más rápida que la fuerza bruta.

Por qué recursividad

Cada llamada decide **un** paso y deja el resto a la siguiente llamada; al regresar, deshace su decisión.

Sudoku

Pon un número; si más adelante choca, bórralo y prueba el siguiente.

Tu horario

Elige grupo para cada materia; si dos chocan, cambia el último que elegiste.

8 reinas

Coloca una reina por fila sin que dos se ataquen.

Las seis estrategias lado a lado

Qué hace cada una, qué gana y dónde hay que tener cuidado.

Estrategia	Idea	Ventaja	Cuidado	Ejemplo
Fuerza bruta	Prueba todo	Siempre encuentra la respuesta	Se vuelve lenta muy rápido	Pares que suman el presupuesto
Divide y vencerás	Parte, resuelve, combina	Rapidez: O(log n), O(n log n)	Los pedazos deben ser independientes	Búsqueda binaria, merge sort
Recursividad	Se llama con un problema más chico	Código corto y claro	Caso base; muchas llamadas	Factorial, carpetas
Programación dinámica	Anota y reutiliza resultados	De $O(2^n)$ a $O(n)$	Usa memoria para la tabla	Fibonacci, corrector, git diff
Voraz	Lo mejor ahora, sin regresar	Muy rápido y simple	No siempre da lo óptimo	Cambio, rutas en mapas
Backtracking	Avanza y regresa si choca	Poda caminos imposibles	Puede seguir siendo lento	Laberinto, Sudoku, horarios

¿Qué tan rápida es cada estrategia? Big O con dos ejemplos

La estrategia no fija el Big O: depende del problema. Pero cada una tiene patrones que se repiten.

Fuerza bruta

Par que suma el presupuesto — $O(n^2)$

revisa todos los pares

Candado de n dígitos — $O(10^n)$

cada dígito multiplica × 10

Divide y vencerás

Búsqueda binaria — $O(\log n)$

descarta la mitad cada vez

Merge sort — $O(n \log n)$

log n niveles, n trabajo en cada uno

Recursividad

calcular_factorial(n) — $O(n)$

una llamada por número

calcular_fibonacci(n) — $O(2^n)$

cada llamada abre otras dos

Programación dinámica

Fibonacci con tabla — $O(n)$

una suma por casilla

Corrector ortográfico — $O(n \cdot m)$

tabla de n × m letras

Voraz

Dar cambio — $O(k + m)$

k tipos de moneda, m monedas

Maleta: valor por kilo — $O(n \log n)$

primero hay que ordenar por valor por kilo

Backtracking

Laberinto de n casillas — $O(n)$

cada casilla se pisa una vez

Sudoku con k vacías — $O(9^k)$

peor caso; la poda ayuda

Señal de alerta: un exponente con n arriba (2^n, 10^n, 9^k) crece tan rápido que con datos grandes nunca termina.

¿Cuál estrategia uso?

Hazte estas preguntas, en orden. La primera que conteste «sí» te da un punto de partida.

1. ¿Hay pocos candidatos?	sí →	**Fuerza bruta**
2. ¿Se puede partir en pedazos del mismo tipo que se resuelven por separado?	sí →	**Divide y vencerás (con recursividad)**
3. ¿Los mismos subproblemas aparecen una y otra vez?	sí →	**Programación dinámica**
4. ¿Elegir lo mejor de ahora nunca te cierra la mejor solución?	sí →	**Voraz**
5. ¿Hay reglas que permiten descartar un camino a la mitad?	sí →	**Backtracking**

Consejo de entrevista: empieza con la fuerza bruta en voz alta («así lo resolvería probando todo») y luego mejórala. Una solución correcta y lenta vale más que una rápida que no funciona.

Práctica

1. ¿Qué estrategia es? Seis situaciones de la vida real.
2. Escribe una función recursiva.
3. Programación dinámica a mano: llena la tabla.
4. ¿El voraz da la mejor respuesta?

Ejercicio 1 · ¿Qué estrategia es?

Lee cada situación y decide cuál de las seis estrategias describe.

a) Olvidaste la combinación del candado de 3 dígitos de tu bici: pruebas 000, 001, 002…

✓ **Fuerza bruta**

b) Buscas a un alumno en la lista ordenada por apellido: abres a la mitad y descartas la mitad que no es.

✓ **Divide y vencerás**

c) Para contar los archivos de una carpeta, cuentas los suyos y repites lo mismo en cada subcarpeta.

✓ **Recursividad**

d) ¿De cuántas formas subes 30 escalones de 1 en 1 o de 2 en 2? Anotas la respuesta de cada escalón y la reutilizas.

✓ **Programación dinámica**

e) Llenas la maleta metiendo primero lo que más vale por kilo, sin volver a sacar nada.

✓ **Voraz**

f) Sudoku: pones un número; si más adelante choca, lo borras y pruebas el siguiente.

✓ **Backtracking**

Ejercicio 2 · Escribe la recursión

calcular_potencia(base, exponente) regresa base elevada a exponente, sin usar ** ni ciclos. El exponente es entero y ≥ 0.

Pistas

- $2^5 = 2 \times 2^4$, y $2^4 = 2 \times 2^3$…
- ¿Cuál es el exponente más chico posible? ¿Cuánto vale cualquier número elevado a él?
- Supón que `calcular_potencia(base, exponente - 1)` ya funciona.

Se espera

```
calcular_potencia(2, 5) → 32
calcular_potencia(7, 0) → 1
```

✓ Una solución

```python
def calcular_potencia(base, exponente):
    if exponente == 0:
        return 1
    return base * calcular_potencia(base, exponente - 1)
```

`calcular_potencia(2, 3)`

```
= 2 * calcular_potencia(2, 2)
= 2 * 2 * calcular_potencia(2, 1)
= 2 * 2 * 2 * calcular_potencia(2, 0)
= 2 * 2 * 2 * 1 = 8
```

Ejercicio 3 · Programación dinámica a mano

Subes una escalera de 1 en 1 o de 2 en 2. ¿De cuántas formas llegas al escalón n?

Idea: al escalón n llegas desde el n – 1 (paso de 1) o desde el n – 2 (paso de 2). Entonces `formas(n) = formas(n – 1) + formas(n – 2)`, con formas(0) = 1 y formas(1) = 1. Copia la tabla y llénala.

n = 0	n = 1	n = 2	n = 3	n = 4	n = 5	n = 6	n = 7	n = 8
1	1	2	3	5	8	13	21	34
		1 + 1	2 + 1	3 + 2	5 + 3	8 + 5	13 + 8	21 + 13

✓ En Python

```python
def contar_formas_escalera(escalones):
    lista_formas = [1, 1]
    for i in range(2, escalones + 1):
        lista_formas.append(lista_formas[i - 1] + lista_formas[i - 2])
    return lista_formas[escalones]
```

Con 30 escalones: **1,346,269** formas, con 29 sumas. Por fuerza bruta habría que enumerarlas todas.

Ejercicio 4 · ¿El voraz da la mejor respuesta?

Sigue a mano la función de las monedas con otro juego de monedas.

Python

```python
def calcular_monedas_cambio(cantidad, lista_monedas):
    lista_cambio = []
    for moneda in lista_monedas:
        while cantidad >= moneda:
            lista_cambio.append(moneda)
            cantidad = cantidad - moneda
    return lista_cambio
```

Llamada

```
calcular_monedas_cambio(11, [9, 6, 5, 1])
```

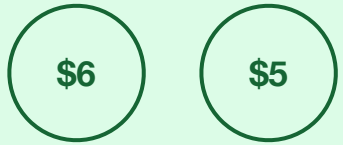
a) ¿Qué regresa?
b) ¿Es el menor número de monedas posible?

a) Regresa [9, 1, 1]: 3 monedas



Después del 9 quedan 2: no caben 6 ni 5.

b) No: [6, 5] son sólo 2 monedas



Para encontrarla hay que revisar opciones: programación dinámica.

Para llevar

Fuerza bruta

Correcta pero lenta. Buena primera versión.

Divide y vencerás

Parte en mitades: O(log n). La lista debe estar ordenada.

Recursividad

Caso base + un problema más chico en cada llamada.

Programación dinámica

Si repites subproblemas, anótalos en una tabla.

Voraz

Rápido, pero comprueba que lo mejor de ahora no estorbe después.

Backtracking

Elegir, explorar, deshacer. Poda lo que rompe las reglas.

Con poco hardware, gana el mejor algoritmo. Antes de programar, pregúntate: ¿pruebo todo, parto, recuerdo, elijo o exploro?